

Module 5: Data Structures

Introduction to C++ Programming — A Beginner's Course

Module Overview

Every variable so far has held exactly one value. This module is the shift from 'one value' to 'many related values' — arrays, the far more practical `std::vector`, strings-as-sequences, and `struct`. The vector-of-structs pattern introduced at the end is worth slowing down for: it is the shape of an enormous amount of real-world data, and it's exactly what the Module 5 project and the course capstone are built on.

Suggested Duration: 75–90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain why array indexing starts at 0 and identify valid index ranges.
- Use `std::vector` to build a list that grows and shrinks at runtime.
- Explain why `std::vector` is generally preferred over plain arrays in this course.
- Treat `std::string` as an indexable, loopable sequence of characters.
- Define and use a `struct` to group related values of different types.
- Build and iterate over a `std::vector` of `struct` instances.

Prerequisites

- Module 4: comfortable with for loops, since every data structure here is looped over.

Materials & Instructor Prep

- `05-data-structures/examples/04_structs.cpp` preloaded — this is the module's capstone-preview example and deserves live walkthrough.

Key Vocabulary

Term	Definition
Array	A fixed-size, ordered collection of values of the same type.
Index	A position in an array/vector/string, starting at 0.
<code>std::vector</code>	A resizable list container from the standard library.
Template	A generic container/type parameterized by another type, e.g. <code>vector<int></code> vs. <code>vector<double></code> .
struct	A user-defined type that groups related fields of possibly different types under one name.
Instance	A specific variable created from a struct (or class) blueprint.

Lesson Outline & Instructor Talking Points

1. Arrays and the zero-index rule (15 min)

- Draw an array of 5 boxes on the board, labeled with indices 0–4. Physically point at box 0 and ask 'is this the first or second element?' — verbally naming 'first' and 'index 0' as the same thing helps it stick.
- Show `scores[5]` on a 5-element array live (if your compiler doesn't crash visibly, explain that this is exactly the danger — it may silently read garbage rather than erroring).

Instructor note: *Zero-indexing is arguably the single most persistent off-by-one source of bugs for the rest of the course — invest real time in the visual/physical framing here.*

2. `std::vector` — the upgrade (20 min)

- Motivate it directly from arrays' limitation: 'What if you don't know in advance how many students will enroll?' Arrays can't grow; vectors can.
- Live-code `push_back` and `pop_back`, printing `.size()` after each call so students watch the container actually change size.
- Show both index-based (`for (int i...)`) and range-based (`for (int x : vec)`) loops over the same vector, and explicitly state when to prefer each (range-based when you don't need the index).

Instructor note: *State the course's default explicitly: 'From here on, default to vector unless you have a specific reason to use a plain array.' Removes an unnecessary decision point for students going forward.*

3. Strings as sequences (10 min)

- Revisit `std::string` from Module 2, now showing `name[0]`, `.length()`, and a range-based for loop over its characters — the point is that a string 'is' a sequence in exactly the same conceptual sense as a vector.

4. struct — grouping different types (20 min)

- Motivate with a concrete need: 'We want to track a student's name, age, and GPA together — but those are three different types. An array or vector alone can't hold mixed types.'
- Build struct `Student` live. Deliberately forget the trailing semicolon after the closing brace once, and show the resulting (often confusing) compiler error — name this specific footgun explicitly.
- Show `s1.name`, `s1.age` member access with the dot operator.

5. The payoff: a vector of structs (15–20 min)

- Build the classroom example live: a `std::vector<Student>`, `push_back` a few, loop over it printing each record.
- Explicitly connect this to the upcoming project and the course capstone: 'This exact pattern — a list of records, each with named fields — is what almost all real-world data looks like: a spreadsheet, a database table, a JSON API response. You just built the C++ shape of that.'

Instructor note: *This is the conceptual peak of the module — protect this time block over any of the earlier ones if the session runs short.*

Common Student Misconceptions

Misconception	How to Address It
"A 5-element array has a valid index 5."	Valid indices are 0 through <code>size-1</code> ; index 5 in a 5-element array is out of bounds and dangerous, not just 'the sixth item.'
"Arrays and vectors are interchangeable in every way."	Arrays are fixed-size; vectors can grow/shrink with <code>push_back</code> / <code>pop_back</code> . Arrays lack these methods entirely.
"A struct needs a semicolon after 'main' like a function does."	A struct definition needs a semicolon after its closing brace precisely because it's a type declaration, not a function — contrast the two forms side by side.

Exercises & Module Project

Exercise: 05-data-structures/exercises/exercise1.cpp — vector statistics (sum/average/min/max) plus a struct `Book` collection. Project: a Student Grade Tracker (05-data-

structures/project/README.md) using a vector of Student structs — this project is the direct seed of the Module 9 capstone.

Facilitation guidance: Tell students explicitly that this project's struct design will be reused (in class form) in the capstone. Naming that connection now motivates careful, clean struct design rather than a rushed one-off.

Discussion & Comprehension Check Questions

1. Why would you choose `std::vector` over a plain array for a shopping cart application?

Guidance: The number of items changes at runtime — vectors support `push_back/pop_back`; arrays have a fixed size decided at compile time.

2. What real-world data have you seen that looks like 'a list of records, each with several named fields'?

Guidance: Contact lists, spreadsheets, database tables, form submissions — anchor the pattern to something familiar to the class.

Assessment Ideas

- Design exercise: given a short word-problem description (e.g. 'track a library's books: title, author, checked-out status'), have students write just the struct definition, no implementation.

Answer Key Notes

See `05-data-structures/exercises/solutions/exercise1_solution.cpp`.

Wrap-Up & Bridge to Next Module

Bridge forward: "You're now duplicating the same average/loop logic in multiple places in your grade tracker. Next module fixes exactly that: functions let you write logic once, name it, and reuse it everywhere."

Instructor Tips

- If the group is strong, mention (briefly, as a forward pointer, not a full lesson) that C++ also has `std::map` and `std::sort` — enough to recognize the names later without derailing this module's scope.